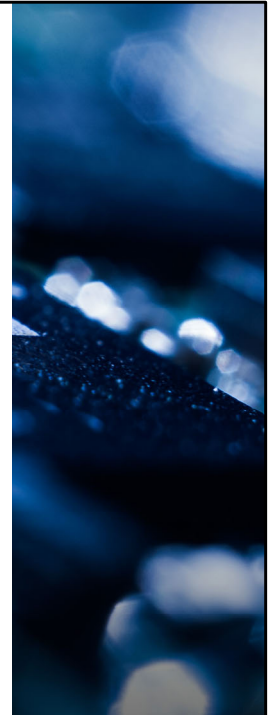




SC1006 Computer Organisation and Architecture

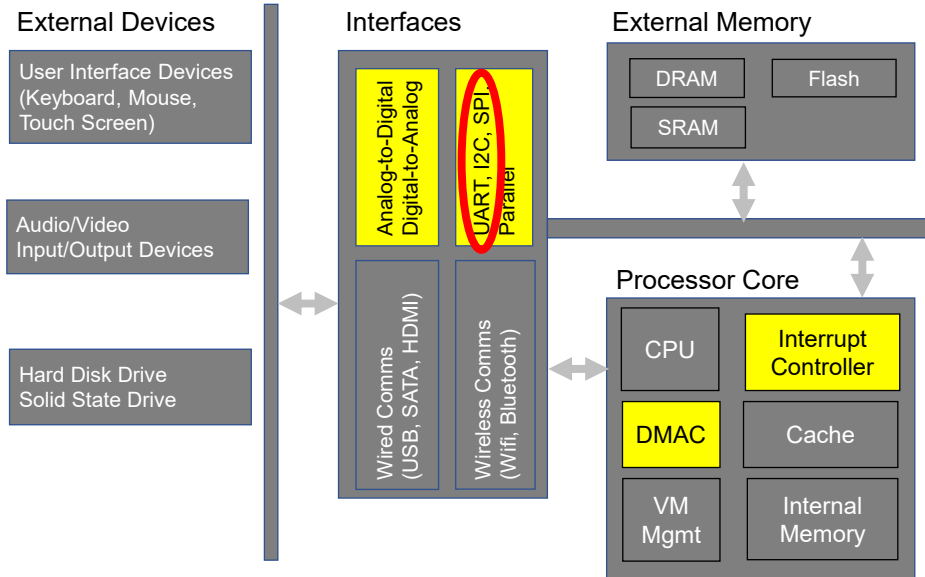
Serial Data Transfer

Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg



- Background picture
<https://pixabay.com/photos/electrician-electronics-computer-1288717/>
- **VO (for approval):**
Next, we will be touching on the Serial Data Interface

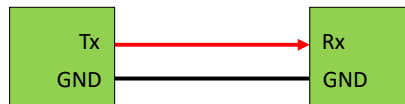
Computer System Block Diagram



VO:

In this video, we will continue, to dive deeper, into these few elements of the computer system.!!

Serial Data Transfer



- Data is transferred **one bit at a time** over a single data line. Comparatively, parallel data interface transfer multiple bits simultaneously.
- **Less affected by signal skew and crosstalk** because there are less electrical wires involved compared to parallel data transfer. Hence, able to support higher frequency clocking.
- Data **transfer rate lower** (compared to parallel interface) given the same clock rate since only one data line is available.

VO:

Serial Data interface, as mentioned earlier, transfer one bit of data at one time. So given the same clock rate, its data transfer rate will be lower than that of the parallel interface.

However, because only one data line per direction is involved,

it is less affected by signal skew since less wires are involved => chance of a skew happening is less.

It is also less affected by crosstalk since there are fewer neighbouring wires or cables as compared to a parallel interface.

With less influence from signal skew and crosstalk, a serial interface bus, if well designed, could potentially achieve a higher clock rate compared to the parallel bus, and this allows its transfer rate to be closer to that of the parallel bus.

Serial Data Transfer Pros and Cons

- Advantage

- Less affected by signal skew and crosstalk because there are less electrical wires involved compared to parallel data transfer. Hence, able to support higher frequency clocking.
- Able to transfer data reliably over a longer distance.
- Lower cost since less wires and connectors are needed.

- Disadvantage

- Data transfer rate lower given the same clock rate since only one data line is available.
- Hardware interface design typically more complex as it need to handle serial to parallel conversion (a processor typically only process in bytes or multiple bytes).

To summarize

(Click)

The Advantages of Serial transfers are

It is less affected by signal skew and crosstalk as there are less wires involved in each interface

As the effect of signal skew and cross talk is not as significant when there are only a few wires, serial interface can therefore be potentially clocked at a higher frequency and is able to transfer data over a longer distance.

It is also less costly since less wires and connectors are needed.

(Click)

The Disadvantages are

Given the same clock rate, Data Transfer rate for serial interface is lower than parallel interface.

Hardware interface design may be more complex because serialization and deserialization operation is needed to convert serial bit data to multiple-bit data format such as word, byte used by a processor.

Classification of Serial Data Transfers

According to

- **Directions of transfers**
- **Methods of synchronization**

- There are 3 types of serial transfer mode:

(Click)

- In Simplex mode, One device is the transmitter while the other is the receiver. Data transfer is only in one direction.

(Click)

- For Half-Duplex mode, Both devices can operate as transmitter as well as receiver. But at any point in time, there is only one transmitter and one receiver, with data flowing in one direction.

(Click)

- Lastly, in Full Duplex mode, Both devices can operate as transmitter as well as receiver. There is separate data line for each direction transfer so each device can transmit and receive data at the same time.

Serial Data Transfer Mode

- **Simplex:** Data transfer in one direction only, with 1 data line.



- There are 3 types of serial transfer mode:

(Click)

- In Simplex mode, One device is the transmitter while the other is the receiver. Data transfer is only in one direction.

(Click)

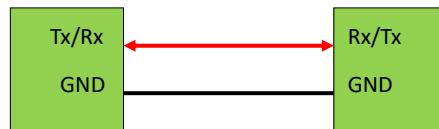
- For Half-Duplex mode, Both devices can operate as transmitter as well as receiver. But at any point in time, there is only one transmitter and one receiver, with data flowing in one direction.

(Click)

- Lastly, in Full Duplex mode, Both devices can operate as transmitter as well as receiver. There is separate data line for each direction transfer so each device can transmit and receive data at the same time.

Serial Data Transfer Mode

- **Half-Duplex:** Data transfer in both directions, but RX and TX is mutually exclusive, also with 1 data line.



- There are 3 types of serial transfer mode:

(Click)

- In Simplex mode, One device is the transmitter while the other is the receiver. Data transfer is only in one direction.

(Click)

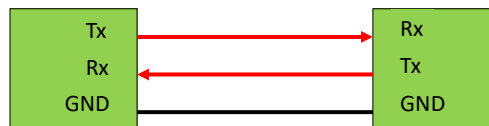
- For Half-Duplex mode, Both devices can operate as transmitter as well as receiver. But at any point in time, there is only one transmitter and one receiver, with data flowing in one direction.

(Click)

- Lastly, in Full Duplex mode, Both devices can operate as transmitter as well as receiver. There is separate data line for each direction transfer so each device can transmit and receive data at the same time.

Serial Data Transfer Mode

- **Full Duplex:** Simultaneous RX and TX, with 2 data lines.



- There are 3 types of serial transfer mode:

(Click)

- In Simplex mode, One device is the transmitter while the other is the receiver. Data transfer is only in one direction.

(Click)

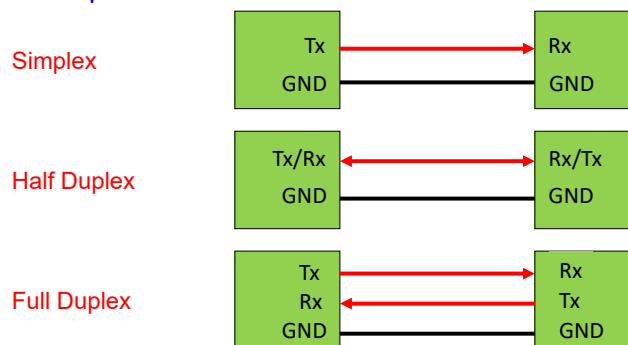
- For Half-Duplex mode, Both devices can operate as transmitter as well as receiver. But at any point in time, there is only one transmitter and one receiver, with data flowing in one direction.

(Click)

- Lastly, in Full Duplex mode, Both devices can operate as transmitter as well as receiver. There is separate data line for each direction transfer so each device can transmit and receive data at the same time.

Serial Data Transfer Mode

- **Simplex**: Data transfer in one direction only.
- **Half-Duplex**: Data transfer in both direction, but RX and TX is mutually exclusive.
- **Full Duplex**: Simultaneous RX and TX



- There are 3 types of serial transfer mode:

(Click)

- In Simplex mode, One device is the transmitter while the other is the receiver. Data transfer is only in one direction.

(Click)

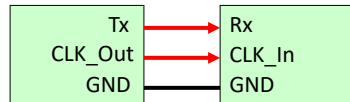
- For Half-Duplex mode, Both devices can operate as transmitter as well as receiver. But at any point in time, there is only one transmitter and one receiver, with data flowing in one direction.

(Click)

- Lastly, in Full Duplex mode, Both devices can operate as transmitter as well as receiver. There is separate data line for each direction transfer so each device can transmit and receive data at the same time.

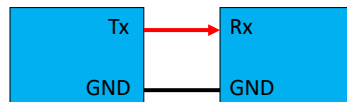
Synchronous vs Asynchronous

Synchronous (similar to parallel interfaces)



- In **synchronous** transmission, there is a **common clock** signal to synchronize the data transfer. E.g. receiver to latch in the data at every rising edge of the clock.

Asynchronous (not available in parallel interfaces)



- In **asynchronous** transmission, there is **no common clock** signal so devices have to agree on a **pre-fixed clock frequency** to use for data transfer.

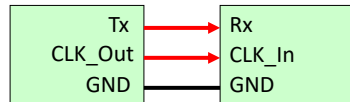
- Serial communications can be Synchronous or Asynchronous in nature and they have different design considerations during implementation.
- A serial communication is synchronous when there is a common clock signal between the transmitter and receiver.
- If a common clock signal is absent, then the transfer is asynchronous in nature.
- For synchronous transmission, the common clock signal is used to synchronize the data transfer, it tells the receiver when to latch in the data bits send by the transmitter.
- For Asynchronous transmission, since there is no common clock. But that doesn't mean that synchronisation is not required between the transmitter and receiver.
- In fact, synchronization is still needed for the data transfer to be successful.
 - Synchronization is achieved by sending synchronization information over the data line, and the devices will have to agree on the clock rate to be used for transferring data.
- This obviously will lead to some issues and we'll go into more detail later.
- Note that this concept of Synchronous and Asynchronous transfer that we discuss

here is referring to the electrical signals layer.

- At higher protocol layer, the word synchronous may take on a slightly different meaning and it is possible to implement synchronous communication over an asynchronous physical link. You need not bother about that now but keep in mind that such implementation is possible.

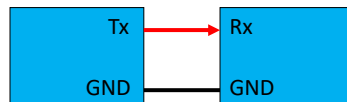
Synchronous vs Asynchronous

Synchronous (similar to parallel interfaces)



- In **synchronous** transmission, there is a **common clock** signal to synchronize the data transfer. E.g. receiver to latch in the data at every rising edge of the clock.

Asynchronous (not available in parallel interfaces)



- In **asynchronous** transmission, there is **no common clock** signal so devices have to agree on a **pre-fixed clock frequency** to use for data transfer.

- Serial communications can be Synchronous or Asynchronous in nature and they have different design considerations during implementation.
- A serial communication is synchronous when there is a common clock signal between the transmitter and receiver.
- If a common clock signal is absent, then the transfer is asynchronous in nature.
- For synchronous transmission, the common clock signal is used to synchronize the data transfer, it tells the receiver when to latch in the data bits send by the transmitter.
- For Asynchronous transmission, since there is no common clock. But that doesn't mean that synchronisation is not required between the transmitter and receiver.
- In fact, synchronization is still needed for the data transfer to be successful.
 - Synchronization is achieved by sending synchronization information over the data line, and the devices will have to agree on the clock rate to be used for transferring data.
- This obviously will lead to some issues and we'll go into more detail later.
- Note that this concept of Synchronous and Asynchronous transfer that we discuss

here is referring to the electrical signals layer.

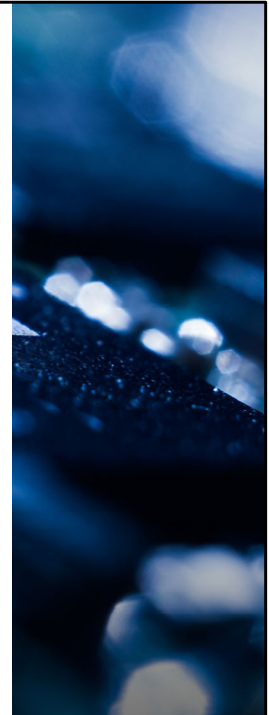
- At higher protocol layer, the word synchronous may take on a slightly different meaning and it is possible to implement synchronous communication over an asynchronous physical link. You need not bother about that now but keep in mind that such implementation is possible.



SC1006 Computer Organisation and Architecture

Synchronous Serial Data Transfer

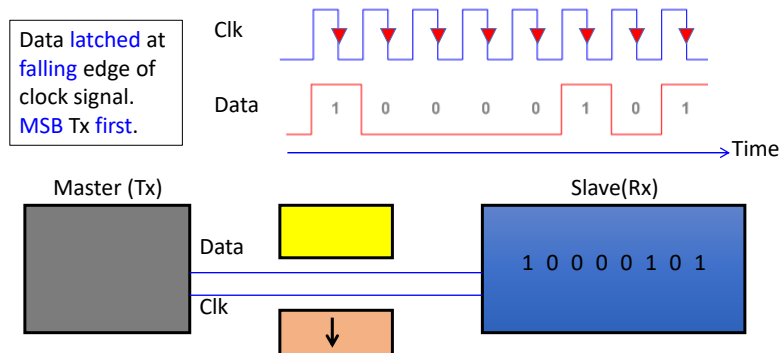
Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg



- Background picture
<https://pixabay.com/photos/electrician-electronics-computer-1288717/>
- **VO (for approval):**
We will be covering synchronous serial transfer in the next few slides.

Synchronous Serial Transfer

- Common clock signal between transmitter and receiver to synchronize the data transfer.
- Master-Slave configuration. With Master providing the clock signal.
- Potentially allows faster transfer rate since no data overhead is needed to synchronize the transfer.



As mentioned, synchronous serial interface has a common clock signal that is used to synchronize the data transfer.

(Click)

The devices are usually configured in a Master-Slave configuration, with master providing the clock.

(Click)

What is shown here is a typical transmission.

The signal waveform is as shown. The typical way of signal transmission is illustrated.

data on the left will be sent/received first.

(Click)

In this example, the data is latched on the falling edge of the clock.

(Click)

The Master will first output a '1' and data will be latched by the slave on the falling edge of the clock signal.

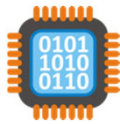
This is followed by a '0' and corresponding clock edge.

Master will continue to send the rest of the data bits, which is latched into the receiver on the falling edge of the clock.

Depending on the configuration, data can also be latched in at the rising edge

instead.

I2C and SPI Bus



- Popular serial buses used by processor to transfer data and control many peripheral devices.

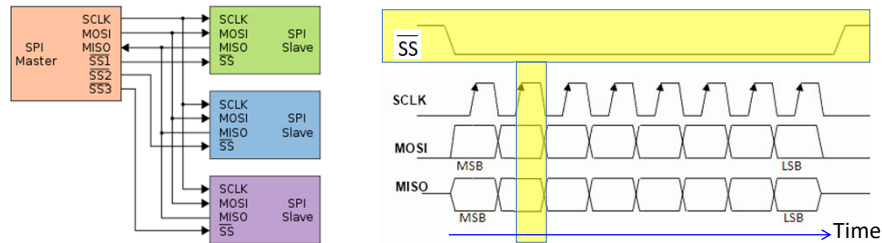
- Accelerometers
- Temperature Sensors
- Touch Screen Controllers
- Power Supply Modules Configuration
- Audio/Video Codecs Configuration



I2C will not be covered in this module

- I2C and SPI are two of the popular synchronous serial bus standards used by a processor for interfacing to other devices.
- Some of the examples are shown here.
- In this course, we will only be touching on SPI bus interface.

Serial Peripheral Interface (SPI) Bus



- To **start** the transfer
 - Slave Select (**SS**) has to be pulled **Low**.
- **Data transfer**
 - MOSI: Master-Out-Slave-In (Master Output, Slave Input)
 - MISO: Master-In-Slave-Out (Slave Output, Master Input)
 - **Data** on MOSI and MISO **latched** in on **rising/falling** clock edge (configurable)
- Allow **multiple slaves** via use of multiple Slave Select.

- SPI bus is a synchronous serial bus. You can see the explicit clock line that the Master used to synchronize transfer over SPI interface.

(Click)

- To initiate a transfer, the master will first pull the Slave Select (SS) pin low. This will enable the target SPI slave.

(Click)

- Data on the MOSI and MISO pins are then latched on the rising edge of the SCLK signal.
- MOSI refers to Master Out Slave In, and this allows master to output data to slave. Similarly, MISO refers to Master In Slave out.

(Click)

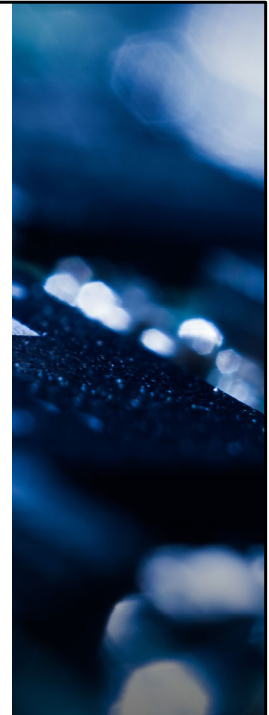
- SPI bus allows multiple slaves to be connected to one master.



SC1006 Computer Organisation and Architecture

Asynchronous Serial Data Transfer

Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg



- Background picture
<https://pixabay.com/photos/electrician-electronics-computer-1288717/>
- **VO (for approval):**
This section is on asynchronous serial interface. This is the last section of the Serial Interface chapter.
- Yes, your agony is going to be over soon.

Asynchronous Serial Transfer

- No common clock is provided between transmitter and receiver.
- Prior to the transmission, the receiver needs to know the transmitting clock rate and the number of bits that are to be transferred with each data packet.
- Special SYNC words are used to indicate START/STOP condition.
- Upon receiving the START SYNC Word, the receiver then uses its own local clock to track the timing.
- Potential skew issue between the two local clocks as transmission progresses.
- Asynchronous Transmission typically also uses SYNC word/bits to provide occasional time stamp for receiver to synchronize its clock to the transmitter clock (helps to reduce clock skew between the two clocks).

- In Asynchronous transfer, as mentioned earlier, there is no common clock between the transmitter and the receiver so they need to rely on other means to achieve synchronization.
- First, both devices need to agree on a certain clock frequency and the number of data bits in a data packet.

(Click)

- They also need a special SYNC word to indicate the start and stop of transmission. This SYNC word is embedded in the data packet. Remember again that there is not an explicit clock line so all information has to be transferred via the data line.
- Upon receiving the Start SYNC word, the receiver will start to track the data using its own local clock. Since the transfer clock frequency is known beforehand, the receiver can roughly guess when the data bits are coming in and latch it.

(Click)

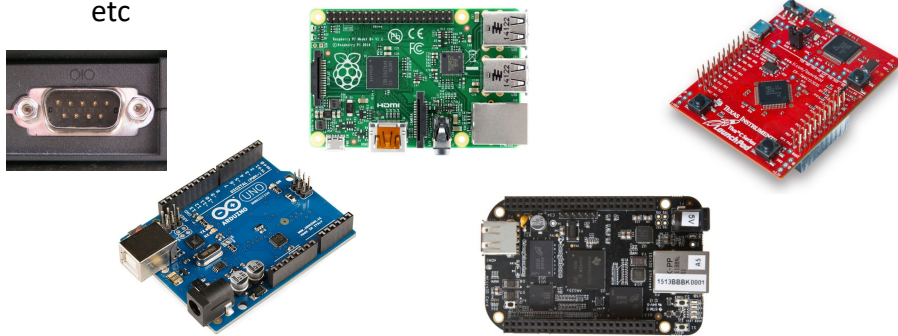
- The fact that the transmitter and receiver are using two different clocks means that over time, these two clocks will drift relative to each other since there is no way these two clocks will run at exactly the same frequency.
- If the clock skew kept increasing, it'll eventually lead to wrong data being latched by the receiver.
- One way to mitigate this issue is for the transmitter to send SYNC word periodically

so that the receiver can use that info to re-align their local clock.

- The re-alignment of clock here means getting the clock edge of the two local clock to occur at the same time stamp.

Universal Asynchronous Receiver Transmitter (UART)

- One of the most commonly used serial interfaces.
 - PC Serial COM Port (RS232) uses UART protocol.
 - USB devices uses a **Virtual COM Port** implementation to connect to the PC.
 - Communication interface between PC and many processor development boards e.g. Arduino Board, TIVA-C Launchpad etc



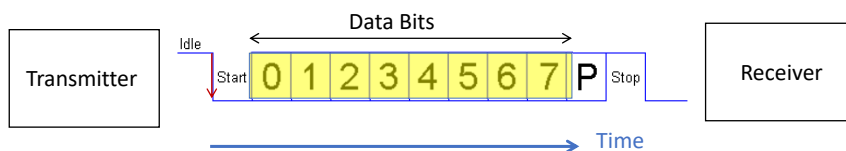
Limit to 3 representative pictures.

One of the most commonly used Asynchronous Serial bus standards is the UART.

- It stands for Universal Asynchronous Receiver Transmitter.
- The PC serial COM Port you see here used to be found in every PC but has since been replaced by USB.
- But UART as a protocol is still a popular protocol for various processor development boards like Arduino and Raspberry Pi.
- You don't see a physical COM Port on these boards though, what you have is the USB interface.
- What happened is that a Virtual COM Port will be enumerated over the USB when processor board such as Arduino is connected to the PC. And the Virtual COM Port uses the UART protocol for communication.

UART Transmit

- During **IDLE** State, the data line is in 'powered' state i.e. Logic '1'.
- The transmitter send a **START** pattern (logic '0') to alert the receiver.
- Sending a logic '0' from idle state (logic '1') will result in a **falling edge** on the data line. This is typically used by the receiver to detect the start of transmission.
- This is followed by the actual **DATA** at a frequency known to the receiver. The transmitting clock rate is also known as the **baud rate**, and determines the number of bits transmitted per second.
- A **PARITY** bit (optional) may also be sent for the receiver to check the integrity of the data packet.
- A **STOP** bit (Logic '1') terminates the transmission.



(Click)

When the UART is IDLE, its data line will be at a logic HIGH.

(Click)

When the transmitter want to send data, it'll first send a STAR bit by pulling the data line LOW.

This falling edge of the data line will alert the receiver to get ready for data reception.

(Click)

The transmitter will then output the actual data at a known frequency, also know as the baud rate (a unit of transmission speed equal to the number of times a signal changes state per second).

(Click)

At the end of the data transfer, the transmitter can optionally output a parity bit. Function of a Parity bit is to enable error detection at the receiver side. We discuss this in more details in the next slide.

(Click)

After that, the transmitter will send a STOP bit to end the transmission.

If the transmitter wants to send another word, it needs to restart the whole process by transmitting the start bit again.

You can't send multiple words with only one set of Start and Stop bit.

For UART protocol, a few of the symbols or status has a fixed logic level

 IDLE state is always Logic '1'

 START bit is always Logic '0'

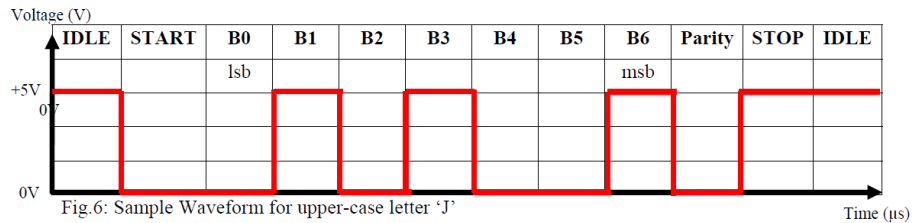
 STOP bit is always Logic '1'

Parity Bit

- If **parity** scheme is **enabled**, the receiver will also sample the parity bit and checks for parity error.
- If **even parity scheme** is used, there should be an **even number of '1's in the data field and the parity field**.
- Hence, if there is an odd number of '1's in the data field, then the parity bit transmitted should be a '1' to make the total even.
- If receiver receives odd number of '1's in a even parity scheme, it'll flag a **Parity Error**, meaning **one or more bits** in the transmission is wrong.
- Vice versa for odd parity scheme.
- The receiver then samples the **STOP** bit(s), if a **'0' is detected instead of '1'**, then receiver will flag a **Framing Error**.

- UART parity scheme is designed to detect errors during transmission
- There are 3 different options: Even, Odd or No Parity.
- If Even Parity Scheme is employed, the transmitter will count the number of '1's when it is transmitting the data. Depending on the number of '1's recorded, it will then transmit a '0' or '1' as the Parity bit, so that the total number of '1's in the Data field and the Parity Field is an Even number.
- For example, if the data transmitted is 1110000, which has three '1's, then the transmitter will transmit a '1' as the parity bit, so that the total number is 4, which is an even number.
- Similarly, Odd Parity scheme implies the transmitter will set or clear the Parity bit so that total number of '1's in data and parity field is ODD.
- No parity means parity bit will not be generated.
- On the receiver side, after it received the data and parity bits from the transmitter, it will count the number of '1's in there, if it doesn't match the parity scheme used, then it implies that one or more bits in the transmission is wrong. But it won't be able to tell the exact bit or bits that are wrong.
- Another type of error is the framing error. This will be flagged if the receiver detects a logic '0' in the bit position where the STOP bit is expected.

UART Tx Example

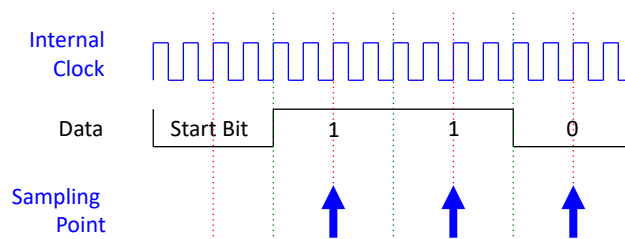


- Figure 6 above correspond to the configuration **7O1** (7 Data, 1 STOP and Odd parity).
- Capital Letter 'J' => Ascii value = 0x4A
- 0x4A => 100 1010 (binary)
- Presented in LSB first => 0101 001
- IDLE=1, START=0, STOP=1.
- Parity bit logic depends on number of 1's in Data Field and the Parity Scheme (Odd or Even) used. It can be of **No (N)** parity check.
- Can practise with **8E1**, **7N1**, **7N2**, ...

- One example to illustrate the UART transmit operation.
- Here we have a **7O1** configuration, which implies 7 Data bits, Odd Parity and 1 STOP bit. There is always only one START bit in each UART frame.
- Notice that for UART transmission, LSB is transmitted first.
- In this example, the data transmitted is 0x4A or 100 1010 in binary. But you'll notice that the waveform shows that the LSB is transmitted first so the logics on the UART signal waveform shows 0101001.
- There are 3 '1's in the data, so the transmitter will clear the parity bit. You can see that Parity bit = 0 in the diagram.

UART Receive

- Receiver monitors the Data line for the **Start Bit**. In real world design, the falling edge on the data line will trigger an interrupt in the microprocessor to start the receiving process.
- Needs to know the **baud rate** in order to **sample the data bits correctly**.
- **Internal clock** of the UART typically run at a **multiple** times (e.g. 16X) of the baud rate so as to time the sampling close to the middle of each data bit.
- Below is an **example of UART receive with internal clock running at 4X** baud rate (for illustration only). Internal clock rate in this example is faster than 4X baud rate in real world implementation.



- let's take a look at the UART receiver operation, which is more complicated than the transmitter.
- The receiver needs to constantly monitor the data line to see if transmitter has anything for it.
- In a real-world design, interrupt mechanism is used to make the process more efficient.

(Click)

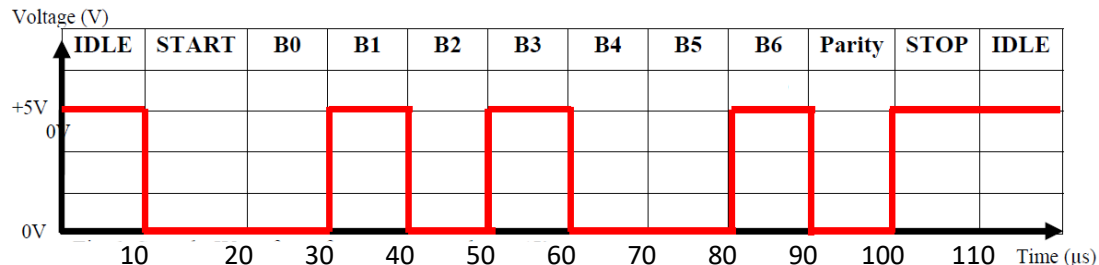
- The receiver needs to know the baud rate in order to sample the data correctly.
- And the internal clock of the UART receiver is typically much faster than the baud rate. One possibility is 16X.
- The receiver will use this faster clock to derive the best location to sample the incoming data more reliably.

(Click)

- The example below illustrates the process. I'm using a 4X sampling here to simplify things. Actual internal clock is usually much faster.
- We have the first logic '0' as the start bit. This is followed by the data bits
- Since the internal clock is running at much faster frequency, the receiver will be able to estimate the mid point of each data bit (for higher reliability) and perform the sampling at that instance.

UART Rx Example

Tx baud rate = $1/(10 \text{ us}) = 100000 \text{ bps}$. Configuration = 7O1.



- RX bits = 0101001b (actual Data=1001010b)
- RX bits = 0011000b always?
- **Possible if Information Sampled @ 200000 bps:**
0001100110000110011
 start → 0101001b

Similarly, let us take a look at the timing diagram example of a UART receiver to illustrate the concepts learnt.

The first Logic '0' is the START bit,

it is followed by the data bits and the bits that comes into the UART receiver in the order of 0101001.

Since LSB is transmitted first, the actual data received is 1001010 or 0x4A.

Odd parity scheme is used so Parity bit as transmitted by the transmitter is '0' so that total number of '1's is an odd number.

Now, looking at the waveform, is it possible for the receiver to receive 0011000 instead?

Answer is yes and that happen when the transmitter and receiver baud rate is different.

In this case, the receiver baud rate is twice that of the transmitter.

So the receiver will be sampling 20 points instead of 10.

It'll then use the UART configuration of 7O1 to decode these 20 samples, as shown in the slide.

The first data received is 0001100. Its LSB first so the actual value is 0011000.